MIDI-TO-TAB: GUITAR TABLATURE INFERENCE VIA MASKED LANGUAGE MODELING

Drew Edwards **Xavier Riley** **Pedro Sarmento** **Simon Dixon**
Centre for Digital Music, Queen Mary University of London, UK
{a.c.edwards, j.x.riley, p.p.sarmento, s.e.dixon}@qmul.ac.uk

ABSTRACT

Guitar tablatures enrich the structure of traditional music notation by assigning each note to a string and fret of a guitar in a particular tuning, indicating precisely where to play the note on the instrument. The problem of generating tablature from a symbolic music representation involves inferring this string and fret assignment per note across an entire composition or performance. On the guitar, multiple string-fret assignments are possible for most pitches, which leads to a large combinatorial space that prevents exhaustive search approaches. Most modern methods use constraint-based dynamic programming to minimize some cost function (e.g. hand position movement). In this work, we introduce a novel deep learning solution to symbolic guitar tablature estimation. We train an encoder-decoder Transformer model in a masked language modeling paradigm to assign notes to strings. The model is first pre-trained on DadaGP, a dataset of over 25K tablatures, and then fine-tuned on a curated set of professionally transcribed guitar performances. Given the subjective nature of assessing tablature quality, we conduct a user study amongst guitarists, wherein we ask participants to rate the playability of multiple versions of tablature for the same four-bar excerpt. The results indicate our system significantly outperforms competing algorithms.

1. INTRODUCTION

Tablatures (tabs) are a type of music notation where each played note is indicated by its physical position on the instrument, as opposed to merely its pitch. Whereas on (e.g.) the piano, each pitch can be played in exactly one location on the instrument, most playable pitches on stringed instruments like the guitar or violin can be played in multiple positions [1]. This redundancy introduces an additional layer of analysis to derive mechanics of a performance from raw pitches. In traditional music scores, e.g. for classical guitar, it is the burden of the performer to select appropriate fingerings and positions for the notes in the sheet music. Similarly, a MIDI transcription of a guitar recording lacks this crucial information for a guitarist to replicate the performance.

In this research we examine the problem of mapping a symbolic representation of a musical performance to guitar tablature. There are few recent publications on this topic (see Section 2), although there are commercial solutions available. Most existing methods propose a manually defined objective function, often related to the difficulty of hand stretches to play chords and distances between hand positions, and seek a solution that minimizes the cost. We take a different approach and provide a modern machine learning treatment of the problem.

We cover the following aspects of our research in this paper: first, we provide a background on the research related to guitar transcription and tablature estimation. Then we formally define the problem of tablature inference from symbolic music notation. Next, we describe the methods of our research, which include: a simple tokenization, a masked language model learning task, a Transformer model solution, pre-training and fine-tuning phases, and a custom beam search inference. Finally, we characterize the performance of our system with quantitative and qualitative metrics, including a detailed user study with 15 guitarists rating various tablatures for short solo guitar excerpts. Our results indicate that guitarists significantly prefer our automatic tablatures versus the commercial alternatives we benchmark.

2. RELATED WORK

The earliest algorithmic approaches to automatic guitar tablature systems date back to Sayegh [2]. His approaches include an expert system approach assigning rules of permissible transitions between hand positions. These are encoded and enforced via Prolog and its native constraint solver. The second approach described assigns costs to transitions between fingerings and uses dynamic programming (Viterbi [3]) to find an optimal path through the constructed weighted graph. This latter approach represents the standard classical benchmark for tablature inference. Alternate approaches include genetic algorithms [4], hybrid expert systems [5], and hidden Markov models [6]. Hori and Sagayama [7] extend the dynamic programming approach by finding a path that minimizes the maximum cost of a local transition across a phrase, as opposed to minimizing the global cost as in Sayegh. Radicioni [8] estimates the fingers employed as well as the fret-string

© D. Edwards, X. Riley, P. Sarmento, and S. Dixon. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** D. Edwards, X. Riley, P. Sarmento, and S. Dixon, "MIDI-to-Tab: Guitar Tablature Inference via Masked Language Modeling", in *Proc. of the 25th Int. Society for Music Information Retrieval Conf.,* San Francisco, United States, 2024.

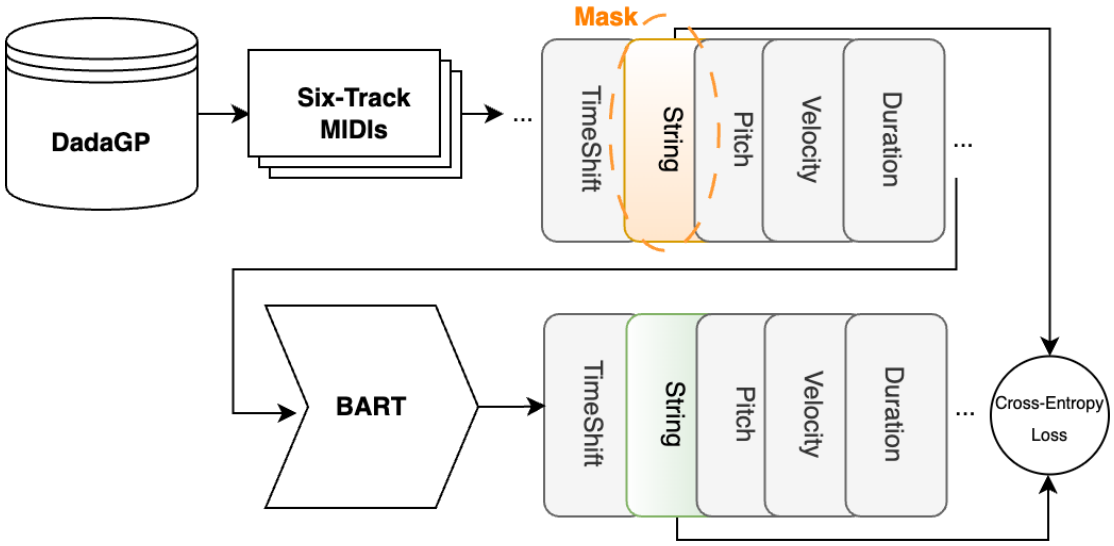


Figure 1. Overview of the training procedure. Guitar Pro files from DadaGP are converted to six-track MIDI files, one file per distinct guitar part and one track per string. These are tokenized into the Structured tokenization of MidiTok. We train a BART model in a simple masked language modeling task where the string tokens are masked out. Only the predictions for the string tokens are used for loss signal propagation.

combinations, using a graph search paradigm to optimize the bio-mechanical comfort of rendering a piece.

In addition to purely symbolic approaches, there is considerable research on the topic of automatic guitar transcription from audio input. Yazawa et al. [9] follow a two-stage approach which uses latent harmonic allocation for multi-pitch estimation (MPE) and then removes unplayable pitches as determined by a fingering cost algorithm similar to Sayegh. Wiggins and Kim [10] apply a convolutional neural network to jointly perform MPE and tablature fingering. The strongest performing MPE methods for guitar [11, 12] leverage the vast amount of transcription material from other instruments (particularly piano) to enlarge the training dataset, but they fall short in offering no tablature estimations.

The most similar approach to our own is described in the master's thesis of Mistler [13], where recurrent neural networks are trained to predict guitar tablatures. However, the training dataset used only contained 74 songs and uses hand-crafted features extracted from the input MusicXML. In contrast, we train on tens of thousands of tabs and process a raw, MIDI-derived tokenization of the input score. The data used for training our network comes from the DadaGP dataset [14], comprising 26,181 song scores in the Guitar Pro format.

3. PROBLEM FORMULATION

We simplify the task of guitar tablature estimation to the task of assigning notes to strings. For a specific guitar tuning, the combination of pitch, string, and fret has only two degrees of freedom. Thus, since the pitch is known a priori, we may predict the string and compute the resulting fret for the assignment. This essentially reduces the problem to *sequence labeling*. In order to increase the flexibility of our system to process a variety of data sources, we begin with MIDI data. Any digital score can be converted to MIDI, and most automatic transcription systems produce MIDI data as well, enabling our tablature system to be composed with any MPE algorithm.

The problem is formally structured as follows: given a one-track MIDI file M, the system produces a six-track MIDI file M_S, where each track contains the notes assigned to a particular string. Let

$$\mathcal{O} = \{64, 59, 55, 50, 45, 40\}$$

denote the list of MIDI note numbers for the open strings of the guitar in standard tuning, corresponding to $E_4, B_3, G_3, D_3, A_2, E_2$, respectively. Thus, to derive the fret of a note with MIDI note number n assigned to a string s, where $s \in \{1, 2, 3, 4, 5, 6\}$, the fret f is calculated by:

$$f = n - \mathcal{O}[s]$$

Although our derivation of the fret value assumes a standard tuning, this approach could be easily modified to permit alternate tunings by changing the values of $\mathcal{O}$.

4. METHODS

4.1 Architecture

Our solution to the problem uses a Transformer with a bidirectional auto-encoder and a left-to-right decoder, based on the BART model architecture [15]. Using such an approach requires a tokenization of the input data. For this, we use the *Structured* tokenization scheme of Huang and Yang [16]. For each MIDI note, we produce five tokens: time shift, string (i.e. track), pitch, velocity, and duration.

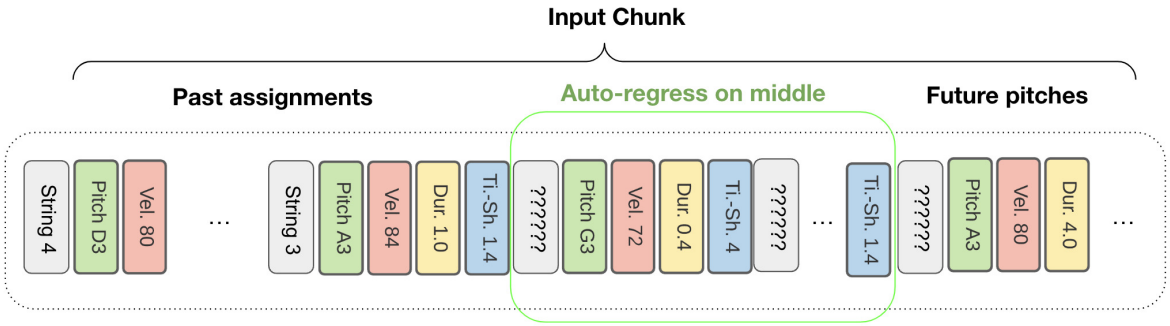


Figure 2. A diagram of our quintile inference algorithm. The middle fifth of the attention window is predicted in an auto-regressive fashion. String assignments from earlier quintiles are fixed. Future notes are available in the context window but will not be assigned until the processing window places them in the center. The beam search is not depicted.

During training, we use a simple masked language modeling supervision scheme, masking the string tokens. This permits both past and future note values to be available for the network during inference, but only past string assignments can be seen. During training, we only compute the loss for string token predictions. We use the Hugging Face Transformers package to define the network, using hyperparmeter settings that simply halve the `bert-base` configuration: 384-dimensional hidden size, 6 hidden layers, 6 attention heads, 1536-dimensional intermediate size, dropout probability of 10%. These hyperparameter values were not finetuned.

4.2 Training

Training takes a two-phase approach (see Figure 1): first we train from scratch on 27,619 guitar tablatures derived from DadaGP [1]. We use the pre-processing code from the SynthTab project [17] to produce a six-track MIDI file for each guitar part in the Guitar Pro files. Training is performed with the AdamW optimizer of PyTorch, employing a linear decay schedule with warm-up and an initial learning rate of 1×10^{-4} and runs for 100 epochs. The second phase of training is a finetuning on precisely annotated guitar performances from the training splits of Riley et al. [12] and GuitarSet [18]. The fine-tuning stage is motivated by the concern of data quality in the DadaGP annotations, which were scraped from the online, crowd-sourced tab library Ultimate Guitar [2]. Here we fine-tune with a learning rate of 1×10^{-5}, again for 100 epochs, on the much smaller data of 281 tabs. Examples are fed into the network in note sequences of length 50, corresponding to 250 tokens per example.

4.3 Inference

A common problem when training Transformer models for auto-regressive tasks is an asymmetry between training and inference regarding previously predicted sequence values. To leverage the parallelism of the Transformer architecture, ground truth labels must be used as decoder input for masked preceding values. However, during inference on unseen data, these labels are unavailable.

We implement a novel inference mechanism for our algorithm. We break up the input segment into quintiles (10 notes or 50 tokens per quintile). Excluding boundary cases, we only make predictions for the center quintile (see Figure 2). This allows our network to have the ability to see the 20 previous note-string assignments and the next 20 future note values. The attention window is advanced by 10 notes per inference step. Additionally, we implement a custom beam search inference. For each string prediction in a quintile, we retain the top two string values for the note. We limit the number of potential paths to 32. The paths are batched to keep inference times nearly equivalent to naive autoregression. Paths are pruned by taking the maximum probability computed by summing the logits of the string predictions. While this does not fully resolve the asymmetry between inference and training, the beam search and additional context provide more probable decoder input values than naive autoregression.

4.4 Post-processing

Thus far we have not imposed any constraints on the output of the network. Ideally, we would take the string predictions and directly augment the score information with the resulting tablature. However, in our qualitative assessment of the system, there are occasions where a string-fret prediction can lead to invalid or unplayable notes. To address these outliers, we attempt to relocate the note to a more suitable string.

The heuristic algorithm is as follows:

1. Merge and sort notes from all strings by start time.
2. Set maximum allowable deviation from the average fret position (*MAX_DEVIATION* = 5) and the highest playable fret (*MAX_FRET* = 21).
3. For each run of 11 notes (5 past, 1 middle, 5 future):
   (a) Find the average fret value of the run, excluding open strings from the computation.
   (b) If the middle note has a fret value exceeding *MAX_FRET* or *MAX_DEVIATION*:

[1] Some pieces in DadaGP have multiple guitar parts, and some tracks are filtered out in the conversion process.

[2] `https://www.ultimate-guitar.com/`

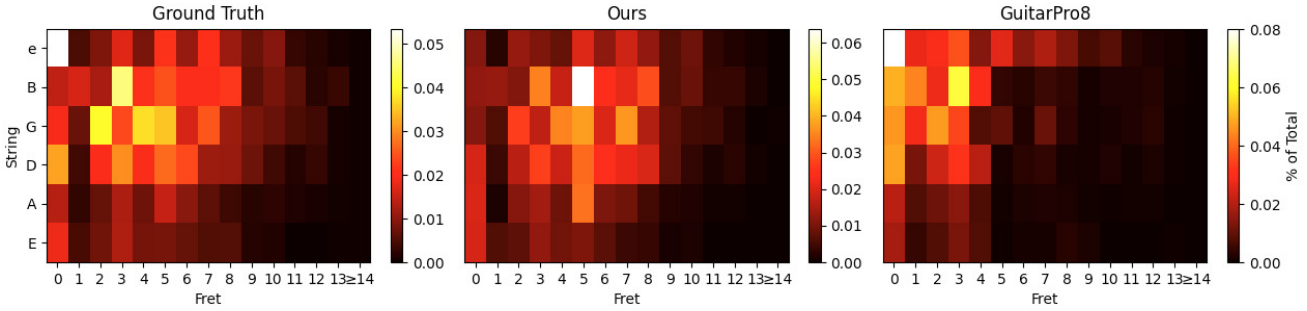


Figure 3. Heatmaps of the fret-string distributions for three of the five tablature systems (ground truth, ours, and Guitar Pro 8). Overall, our system has a similar distribution to the ground truth, but the output appears to be biased away from open strings. Guitar Pro 8 shows a heavy skew to low frets, which perhaps suggest a bias towards playing in "first position" (playing primarily on frets 1 to 4).

i. Define the available strings to be those with no notes intersecting with the note under consideration.
ii. If an open string is available, select it.
iii. Else, select the string yielding a fret value closest to the neighborhood mean.

Ultimately, in our test set from Riley et al. [12], our post-processing algorithm only modifies 0.53% of all notes. However rare, addressing these failures is important to ensure the resulting tab is playable.

5. RESULTS

5.1 Quantitative Results

We use the training splits from Riley et al. [12], with 61 pieces in the training set, 8 in the validation set, and 9 in the test set, corresponding to 58,080, 7,031, and 8,451 notes respectively. At the end of finetuning, our next note accuracy on the validation set is 94.35%. This is the probability of correctly inferring the next note-string assignment given ground truth labels up to the point of prediction. When evaluating autoregressively on the held-out test set across 50-note [3] examples, our network agrees with ground truth on 82.52% of predictions. This discrepancy highlights the difference between teacher-forcing and errors accumulated in auto-regressive inference. We measure the impact of the finetuning step by evaluating the pre-trained model without finetuning, which gives 78.48% agreement, corresponding to a 4.04 percentage point difference due to finetuning.

To compare our algorithm to existing technologies, we use one commercially available and two open-source implementations of automatic tablature systems. Guitar Pro 8 [4] is a music software program designed for editing, visualizing, and sharing guitar, bass, and other stringed instruments' tablatures, and includes an algorithm to automatically produce tablature from score or MIDI. MuseScore [5] is an open-source score editor with a similar functionality for generating tablature. TuxGuitar [6] is free, open-source software for creating and playing guitar tablature and standard musical notation. We use each of these systems to generate MusicXML files with tablature for our 9 held-out test scores from our finetuning dataset, which are then used for evaluation.

Objective evaluation of guitar tablature is difficult, as we will discuss further in Section 5.2. We provide three metrics that illustrate the strength of our system. The first metric is a measure of agreement between the ground truth note-string assignment and each algorithm's assignment for the corresponding note. The metric is computed by matching [7] notes from each measure of the ground truth with the notes from the inferred tablature's corresponding measure. An agreement occurs when the ground truth and the inferred tab assign the same string. The total number of agreements is counted across all examples and then divided by the total number of notes compared. Our system shows the highest agreement of 73.18% (see Table 1). This falls below our 84.42% agreement from the 50-note examples, because early disagreements on fretboard location for a group of notes will likely cause subsequent note assignments to continue to disagree.

The other two metrics relate to the "*stretch*" values across chords in the MusicXML. Chords are extracted as note onsets occurring at the exact same time. For all such groups of notes, we define the stretch as the maximum fret-wise distance between any two notes in the chord. For example, a chord with notes F3, C4, E4, and A4 played on the string-fret locations [8] (4, 3), (3, 5), (2, 5), (1, 5) will have a stretch value of 2. Open strings do not restrict hand positions so they do not contribute to the stretch. We report the maximum and average stretch across the chords in the test set. The ground truth has the lowest maximum stretch of 6. Our system demonstrates occasional erratic behavior of assigning high notes to lower strings, resulting in a shifted mean and larger maximum stretch value of 12. In Figure 4, we compare frequencies of maximum fret distances between our system and the ground truth. An example failure is shown in Figure 5. The mean and median values indicate that, in general, all algorithms attempt to place chords

[3] Recall the model has a context window of 50 notes.
[4] https://www.guitar-pro.com/
[5] https://musescore.com/
[6] https://www.tuxguitar.app/
[7] Matching is required due to reordering of simultaneous notes.
[8] String 1 is the High E string, and fret values start at 0 for open strings.

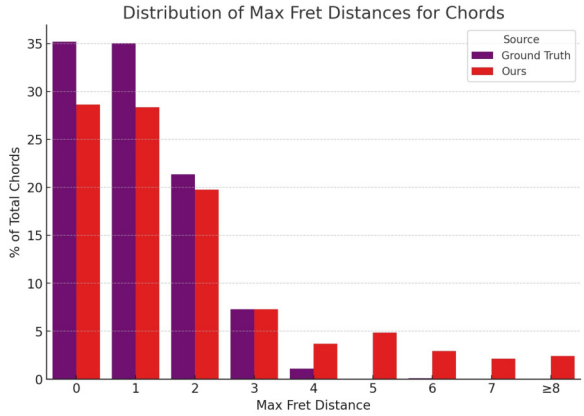


Figure 4. Comparison of the distributions of stretch distances between chords in the test set.

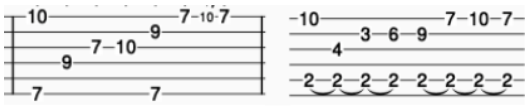


Figure 5. An example failure of our system. Ground truth is left, ours is right. The assignment of B2 to the fifth string creates an 8-fret stretch, which is essentially unplayable.

within a narrow band of frets. The presence of these large stretches motivates future work to better inform our algorithm about the importance of physical playability.

As a final quantitative comparison, we compute fret-string distributions for all five sets of tablatures. For each distribution, we compute the Kullback–Leibler divergence against the ground truth distribution. Our system has the lowest value of 0.099; the other values are: 0.462 for Guitar Pro 8, 0.635 for MuseScore, and 1.286 for TuxGuitar. Three of these distributions are shown in Figure 3.

5.2 User Study

A purely quantitative evaluation of automatic guitar tablature systems is problematic because there may be multiple ways to play the same phrase or excerpt of music. For example, in Figure 6, we show two distinct tablatures for the same one-bar phrase. The top is the ground truth transcription and the bottom is the layout from our system. At a glance, both provide reasonable fretboard fingerings. The ground truth shows a preference for open strings, but our system better minimizes the maximum span between successive notes (2 frets versus 4 frets). However, in this example, our system only agrees with the ground truth on two notes, which corresponds to an accuracy of 16.67%. On the other hand, hand-crafted metrics (such as maximum or average span between notes) fail to capture complex preferences of guitar tablature – otherwise existing systems that minimize these values as cost functions would suffice.

Table 1. Summary of quantitative analysis, showing maximum, mean and median "stretch" of chords in the tablature, defined as the maximum fret distance between any two notes in the chord. We also report the percent agreement with the ground truth note-string assignment. All metrics are averaged over the entire withheld test set.

Source	Max Stretch	Mean Stretch	Median Stretch	% Agree
Ground Truth	6	1.04	0	–
Ours	12	1.84	1	73.58
MuseScore	10	1.19	1	62.51
Guitar Pro 8	12	0.78	0	62.27
TuxGuitar	18	2.03	1	55.42

To complement the quantitative analysis and circumvent some of the potential limitations of the approach, we conducted a study to assess guitarists' opinions on the *playability* and overall preferences for tablatures. Participants were exposed to 30 audio excerpts consisting of 4-bars of solo jazz guitar audio, and for each were shown 5 distinct tablature transcriptions from the following groups: TuxGuitar (*TG*), MuseScore (*MS*), Guitar Pro 8 (*GP*), our system (*Ours*) and ground truth (*GT*), which was created by a professional transcriber. The stimuli were selected by randomly sampling the test split of Riley et al. [12]. Via an online listening study, we probed how guitar players deem the tablatures generated by our system, and how they rank them against the ground truth and the outputs from other tablature generation software (i.e. *TG*, *MS* and *GP*).

The online listening study took approximately 1.5 hours to complete and both the order of audio excerpts and the order of tablature transcriptions were randomized. As conditions to take part in the study we proposed that participants should be guitar players and have a familiarity with reading tablatures, access to headphones or speakers and normal hearing. Subjects were instructed to ignore the difficulty of the music excerpts as they rated the playability of the tablatures. Overall, we recruited 15 guitarists and invited them to attempt to play each of the tablature examples on the guitar during the study, while rating each of the tablature transcription groups on a scale from 1 to 10. Participants, with an age distribution of 39 ± 14 years, reported a median value of 10 years of daily regular engagement with practice of the guitar, and a median value of 3 hours of guitar practice per day at the peak of their interest. The study received ethical approval from the Queen Mary University of London Ethics of Research Committee (QMERC20.565.DSEECS24.012), and participants were compensated with an Amazon gift voucher. Results for the listening test can be observed in Figure 7.



Figure 6. Two tablatures for the same musical excerpt. The top is ground truth, the bottom is from our system.

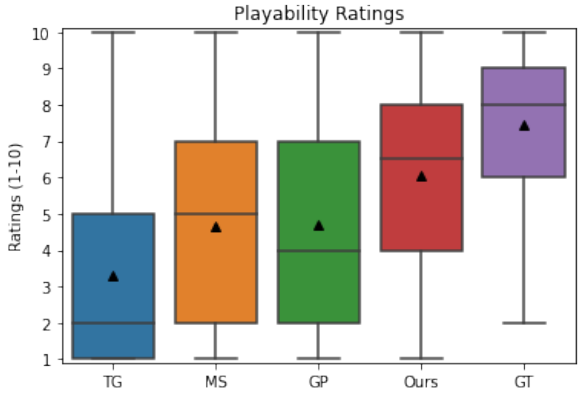


Figure 7. Box plots of the results of the listening study on the playability of tablatures. Bars indicate median values and triangles indicate mean values, for each group.

As expected, the ground truth group ranks highest (7.45 ± 2.62). We hypothesise that the reason why the ground truth falls well short of a "perfect" score is linked to the subjective preferences of participants in terms of fingerings and note position choices, which are inherently linked to their guitar playing techniques and overall style. Furthermore, the ground truth represents professional transcriptions of jazz recordings, where perfect information of the original tablature is not available. As discussed above and illustrated in Figure 6, there can be multiple reasonable ways to play a phrase, and we observed that the ratings for our system were higher than those of the ground truth group for 101 ratings out of 450 in total. The results show that participants tend to rate the playability and overall preference of the tablatures from our system (6.04 ± 2.67) higher than the ones from the competing software (*TG*: 3.32 ± 2.87; *MS*: 4.67 ± 2.88; *GP*: 4.69 ± 2.64).

Of the 2,250 data points collected (30 excerpts $\times$ 5 tabs $\times$ 15 participants), a Shapiro-Wilk test showed that data was not normally distributed within groups. Due to the repeated measurements characteristic of the test (every participant rates all the stimuli), we use a Friedman test to investigate the effects of the type of tablature transcription system on the perceived playability of tablatures, with a Type I error α of 0.05. The statistical results showed a highly significant effect of the tablature transcription system in the participants' responses amongst groups ($\chi^2(4) = 532.09$, $p < .001$). Finally, in order to determine if there were statistically significant differences between groups, we conducted a post-hoc pairwise Wilcoxon test, Bonferroni-adjusted α level of 0.005 $(.05/10)$. This yielded highly significant differences in ratings between groups, except for (*MS*, *GP*).

6. DISCUSSION

Our results suggest a data-driven approach to guitar tablature inference can yield predictions that are significantly more aligned with guitarists' preferences than existing methods. These results are very encouraging given the simplicity of our approach. Our system imposes no constraints on the predicted tablatures until the final post-processing, during which less than 1% of note-string assignments are modified. Future research in this direction may benefit from more directly encoding positional fretboard locations and physical limitations as input to the network.

Despite the strong results, our system has several limitations to be addressed. Guitar tuning is never explicitly encoded as input to the model. Since we only predict string values, our fret predictions are always derived from the note-string assignment and an assumption of standard tuning. Similarly, our system does not handle the use of capos. The system is unaware of many guitar specific articulations, such as harmonics, hammer-ons, pull-offs, and pitch bends. Finally, we make no attempt to assign individual notes to the fingers of a guitarist, which is occasionally done in professional scores or transcriptions, and would be a necessary step in order to estimate playability explicitly.

Another criticism of our approach is that it does not use visual and audio cues for fretboard prediction. As shown by Bastas et al. [19], inharmonicity analysis of a particular instrument can improve string predictions. Likewise, Duke and Salgian [20] demonstrate how computer vision models can be used for accurate and real-time tablature transcription. Both of these directions of research offer a more faithful reproduction of a particular performance, since a symbolic approach simply has no access to disambiguating signals such as hand position or string inharmonicity. However, this shortcoming can also be viewed as a strength: our system does not need access to video nor audio. From this perspective, our approach can be viewed as an automatic arranging system for guitar tablature performance.

The main failure mode of our system is the assignment of unplayable chords at a small but significant frequency (2.4% of chords have a maximum fret distance exceeding 7). Future research may explore different tokenization schemes: encoding fret values as input, physically inspired loss functions, or more carefully designed post-processing to handle these cases. However, the vast majority of the mass of the distribution of chord stretch distances falls within playable limits, which indicates that the algorithm is implicitly modeling some of the physical constraints that classical systems use to derive tablatures.

7. CONCLUSION

We present a deep learning algorithm to predict guitar tablature from symbolic music notation. Our methodology trains an encoder-decoder Transformer to learn tablature assignment from raw note events. Drawing inspiration from natural language processing, we begin by pretraining on a dataset of tens of thousands of tablatures and then fine-tune on a curated dataset of professional guitar scores. We evaluate our system against commercially available software and demonstrate a significant preference for our system through a user study among guitarists. Our MIDI-to-Tab system represents a first step towards achieving human-level tablature inference via machine learning.

8. ACKNOWLEDGMENTS

Authors DE, XR, and PS are research students at the UKRI Centre for Doctoral Training in Artificial Intelligence and Music, supported by UK Research and Innovation [grant number EP/S022694/1] and Yamaha Corporation (DE).

9. REFERENCES

[1] P. Sarmento, A. Kumar, D. Xie, C. Carr, Z. Zukowski, and M. Barthet, "ShredGP: Guitarist style-conditioned tablature generation," in *The 16th International Symposium on Computer Music Multidisciplinary Research*, Tokyo, Japan, 2023.

[2] S. I. Sayegh, "Fingering for string instruments with the optimum path paradigm," *Computer Music Journal*, vol. 13, no. 3, pp. 76–84, 1989. [Online]. Available: http://www.jstor.org/stable/3680014

[3] A. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260–269, 1967.

[4] D. R. Tuohy and W. D. Potter., "A genetic algorithm for the automatic generation of playable guitar tablature," in *Proceedings of International Computer Music Conference,*, 2005, p. 499–502.

[5] M. Miura, I. Hirota, N. Hama, and M. Yanagida, "Constructing a system for finger-position determination and tablature generation for playing melodies on guitars," *Systems and Computers in Japan*, vol. 35, no. 6, pp. 10–19, 2004. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/scj.10609

[6] G. Hori, H. Kameoka, and S. Sagayama, "Input-output HMM applied to automatic arrangement for guitars," *Journal of Information Processing*, vol. 21, pp. 264–271, 04 2013.

[7] G. Hori and S. Sagayama, "Minimax Viterbi algorithm for HMM-based guitar fingering decision," in *17th International Society for Music Information Retrieval Conference*, 2016. [Online]. Available: https://api.semanticscholar.org/CorpusID:10478009

[8] D. Radicioni, "Computational modeling of fingering in music performance," Ph.D. dissertation, Università di Torino, Centro di Scienza Cognitiva, 2005.

[9] K. Yazawa, K. Itoyama, and H. G. Okuno, "Automatic transcription of guitar tablature from audio signals in accordance with player's proficiency," in *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2014, pp. 3122–3126.

[10] A. Wiggins and Y. Kim, "Guitar tablature estimation with a convolutional neural network," in *Proceedings of the 20th International Society for Music Information Retrieval Conference*. ISMIR, Nov. 2019, pp. 284–291. [Online]. Available: https://doi.org/10.5281/zenodo.3527800

[11] W. T. Lu, J. Wang, and Y. Hung, "Multitrack music transcription with a time-frequency perceiver," *CoRR*, 2023, abs/2306.10785.

[12] X. Riley, D. Edwards, and S. Dixon, "High resolution guitar transcription via domain adaptation," in *2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 1051–1055.

[13] E. Mistler, "Generating guitar tablatures with neural networks," Master's thesis, The University of Edinburgh, School of Informatics, 2017.

[14] P. Sarmento, A. Kumar, C. Carr, Z. Zukowski, M. Barthet, and Y.-H. Yang, "DadaGP: A dataset of tokenized GuitarPro songs for sequence models," in *Proc. of the 22nd Int. Soc. for Music Information Retrieval Conf.*, 2021, pp. 610–618.

[15] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A.-R. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Annual Meeting of the Association for Computational Linguistics*, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:204960716

[16] Y.-S. Huang and Y.-H. Yang, "Pop Music Transformer: Beat-based modeling and generation of expressive pop piano compositions," in *Proceedings of the 28th ACM International Conference on Multimedia*. New York, NY, USA: Association for Computing Machinery, 2020, p. 1180–1188. [Online]. Available: https://doi.org/10.1145/3394171.3413671

[17] Y. Zang, Y. Zhong, F. Cwitkowitz, and Z. Duan, "SynthTab: Leveraging synthesized data for guitar tablature transcription," in *2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024.

[18] Q. Xi, R. M. Bittner, J. Pauwels, X. Ye, and J. P. Bello, "GuitarSet: A dataset for guitar transcription," in *Proceedings of the 19th International Society for Music Information Retrieval Conference*, Paris, France, 2018, pp. 453–460.

[19] G. Bastas, S. Koutoupis, M. Kaliakatsos-Papakostas, V. Katsouros, and P. Maragos, "A few-sample strategy for guitar tablature transcription based on inharmonicity analysis and playability constraints," in *2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 771–775.

[20] B. Duke and A. Salgian, "Guitar tablature generation using computer vision," in *Advances in Visual Computing: 14th International Symposium on Visual Computing (ISVC 2019), Proceedings, Part II*. Berlin, Heidelberg: Springer-Verlag, 2019, p. 247–257. [Online]. Available: https://doi.org/10.1007/978-3-030-33723-0_20